

CP2 - CI/CD com GitHub Actions e Argo CD

Relatorio de entrega

Integrantes:

Edmario Santos - RM565486

Kaue Lima - RM562071

Repositorio GitHub: <https://github.com/edmariosantos92/cp2-cicd-argocd>

Ambiente utilizado: Minikube local (driver Docker), Windows 11

Data: 21/09/2026

1. Ambiente utilizado

Foi utilizado um cluster Kubernetes local criado com Minikube (driver docker), rodando em uma maquina Windows 11 com Docker Desktop. Essa opcao foi escolhida por nao exigir conta em nuvem, permitindo reproduzir o fluxo completo de CI/CD e GitOps localmente. A aplicacao foi exposta atraves de NodePort, acessado via 'minikube service'.

2. Funcao do GitHub Actions

O GitHub Actions e responsavel pela parte de Integracao Continua (CI) do fluxo: a cada push que altera a pasta app/ (codigo da aplicacao), um workflow (.github/workflows/ci.yml) e disparado automaticamente. Esse workflow faz o checkout do codigo, builda a imagem Docker da aplicacao, autentica no GitHub Container Registry (ghcr.io) usando o token automatico do proprio Actions (GITHUB_TOKEN), publica a imagem com duas tags (o SHA curto do commit e 'latest'), e por fim atualiza a tag da imagem dentro do manifesto k8s/deployment.yaml, commitando essa mudanca de volta no repositorio.

3. Container Registry utilizado

Foi utilizado o GitHub Container Registry (ghcr.io), pela integracao nativa com o GitHub Actions (autenticacao automatica via GITHUB_TOKEN, sem necessidade de criar credenciais/secrets manualmente) e por herdar a visibilidade publica do repositorio, dispensando configuracao de imagePullSecrets no cluster.

4. Funcao do Argo CD

O Argo CD e responsavel pela parte de Entrega/Implantacao Continua (CD): ele monitora continuamente a pasta k8s/ do repositorio Git e mantem o estado do cluster Kubernetes sincronizado com o que esta declarado nos manifestos daquela pasta. Foi configurado com sincronizacao automatica (syncPolicy.automated), incluindo 'selfHeal' (reverte qualquer mudanca manual feita direto no cluster) e 'prune' (remove recursos que saem do repositorio). Assim, nenhum 'kubectl apply' manual e necessario apos a configuracao inicial - o Argo CD aplica sozinho qualquer mudanca commitada em k8s/.

5. O que significa GitOps

GitOps é uma prática de operação de infraestrutura e aplicações em que o repositório Git é a única fonte de verdade (single source of truth) do estado desejado do sistema. Em vez de aplicar mudanças diretamente no cluster, todas as alterações são feitas via commits/pull requests no Git, e um agente (no caso, o Argo CD) observa o repositório e reconcilia automaticamente o estado real do cluster com o estado declarado nos manifestos versionados. Isso traz rastreabilidade (todo o histórico de mudanças fica no Git), reversibilidade (um rollback e apenas um 'git revert') e auditoria.

6. Diferença entre CI e CD nesta atividade

CI (Integração Contínua) é feito pelo GitHub Actions: cobre a parte de build e publicação - compilar a aplicação em uma imagem Docker, testar (nesta atividade, validar que o build/publish funcionam) e publicar essa imagem em um registry, além de atualizar a referência da imagem no manifesto. CD (Entrega/Implantação Contínua) é feito pelo Argo CD: cobre a parte de deploy - detectar que o manifesto no Git mudou e aplicar essa mudança no cluster Kubernetes real, garantindo que o ambiente rodando sempre reflita o que está declarado no repositório. Ou seja, o CI termina quando a imagem está publicada e o manifesto atualizado no Git; o CD começa a partir daí, levando essa mudança até o cluster.

7. O que aconteceu depois que a alteração foi enviada ao GitHub

Ao dar 'git push' com a alteração (versão da aplicação de 1.0.0 para 1.1.0 e mudança da cor de fundo), o GitHub Actions detectou o push na branch main tocando a pasta app/ e disparou o workflow de CI automaticamente. O workflow buildou uma nova imagem Docker, publicou no GHCR com uma nova tag baseada no SHA do commit, e commitou de volta a atualização da tag no k8s/deployment.yaml. O Argo CD, monitorando o repositório, detectou que o manifesto mudou, marcou a aplicação como 'OutOfSync' e, graças à sincronização automática configurada, aplicou a mudança sozinho: um novo Pod foi criado com a nova imagem e o Pod antigo foi removido (rolling update). Poucos segundos depois, a aplicação acessada pelo navegador já refletia a nova versão e a nova cor, sem nenhuma intervenção manual no cluster.

8. Dificuldades encontradas e soluções

Visibilidade do pacote no GHCR: havia a expectativa de precisar tornar o pacote público manualmente nas configurações do GitHub, mas ao publicar a imagem pela primeira vez o pacote já herdou a visibilidade pública do repositório automaticamente, então nenhuma ação adicional foi necessária.

CRD do Argo CD excedendo limite de anotações do kubectl: a instalação padrão do Argo CD via 'kubectl apply' falhou em uma das CustomResourceDefinitions com o erro 'annotations: Too long'. A solução foi reaplicar a instalação usando 'kubectl apply --server-side', que evita a anotação last-applied-configuration responsável pelo limite.

Variáveis de ambiente sobrescrevendo a versão da imagem: na primeira tentativa de demonstrar a atualização automática, o manifesto k8s/deployment.yaml fixava APP_VERSION e APP_COLOR como variáveis de ambiente do container, o que sobrescrevia os valores definidos dentro da própria imagem Docker (lidos do package.json/server.js). Isso fazia a aplicação continuar mostrando a versão antiga mesmo após o deploy da nova imagem. A correção foi remover essas variáveis fixas do manifesto, deixando a aplicação usar os valores definidos na imagem publicada - e assim demonstrar de fato o efeito do pipeline.

Certificado autoassinado do Argo CD: o acesso à interface web do Argo CD via 'https://localhost:8080' exibia um aviso de conexão insegura por usar certificado autoassinado, exigindo confirmação manual no navegador para prosseguir (comportamento esperado em ambiente local sem certificado válido).

Evidencias

Capturas de tela e saídas de comando organizadas na ordem do fluxo executado.

1. Cluster Kubernetes funcionando

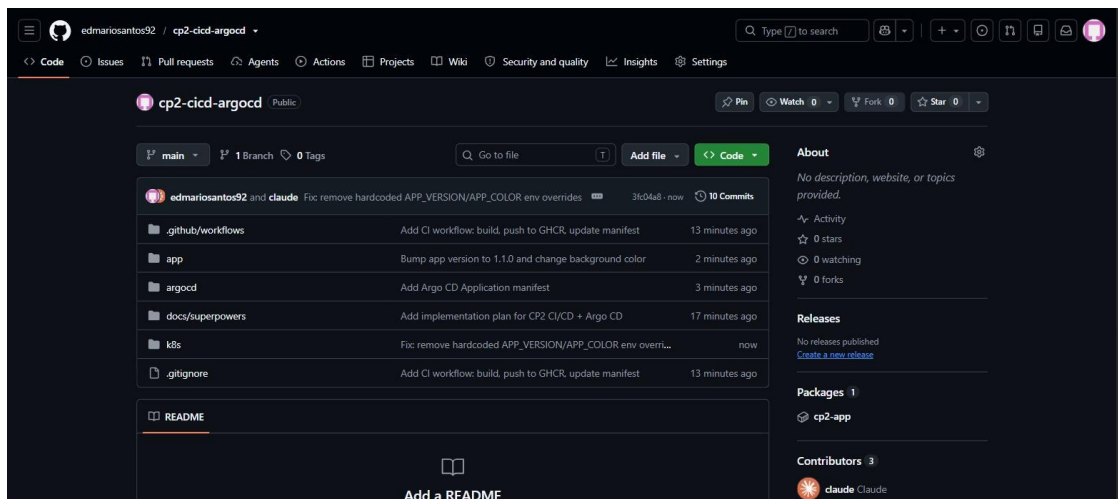
kubectl get nodes / cluster-info - node minikube em Ready

```
$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready    control-plane   14m   v1.37.0

$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:50270
CoreDNS is running at
https://127.0.0.1:50270/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

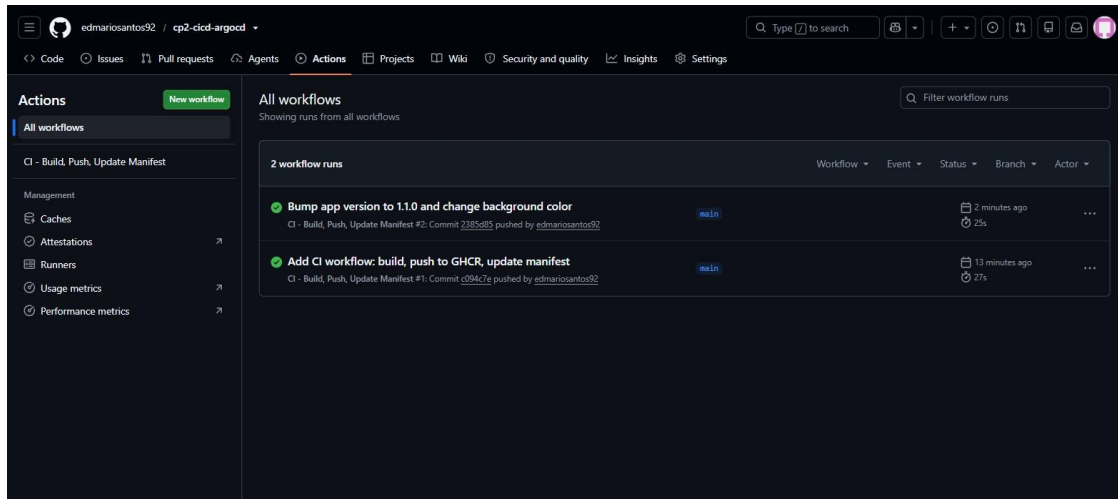
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

2. Repositorio GitHub



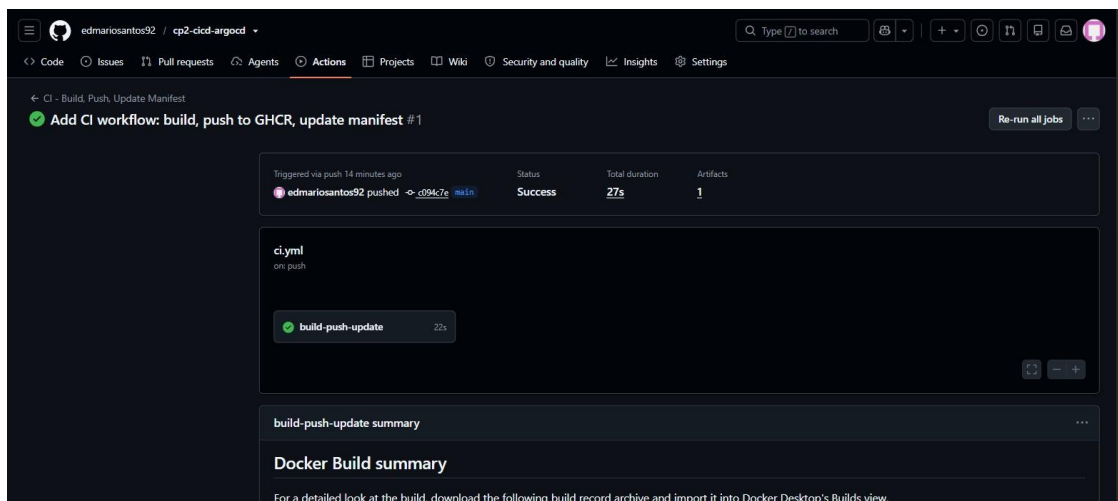
Repositorio edmariosantos92/cp2-cicd-argocd com app/, k8s/ e .github/workflows/

3. Pipeline do GitHub Actions



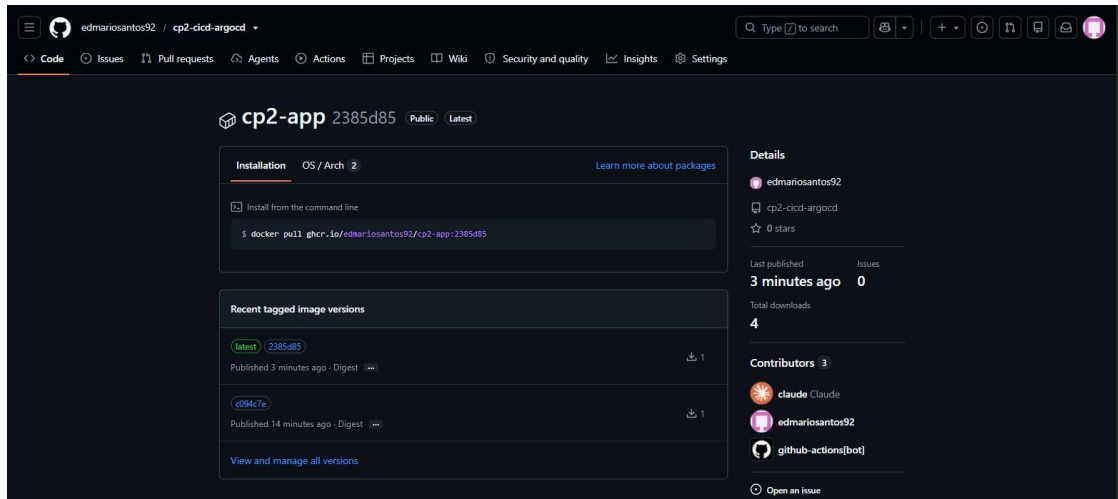
Lista de execucoes do workflow CI - Build, Push, Update Manifest

4. Pipeline executada com sucesso



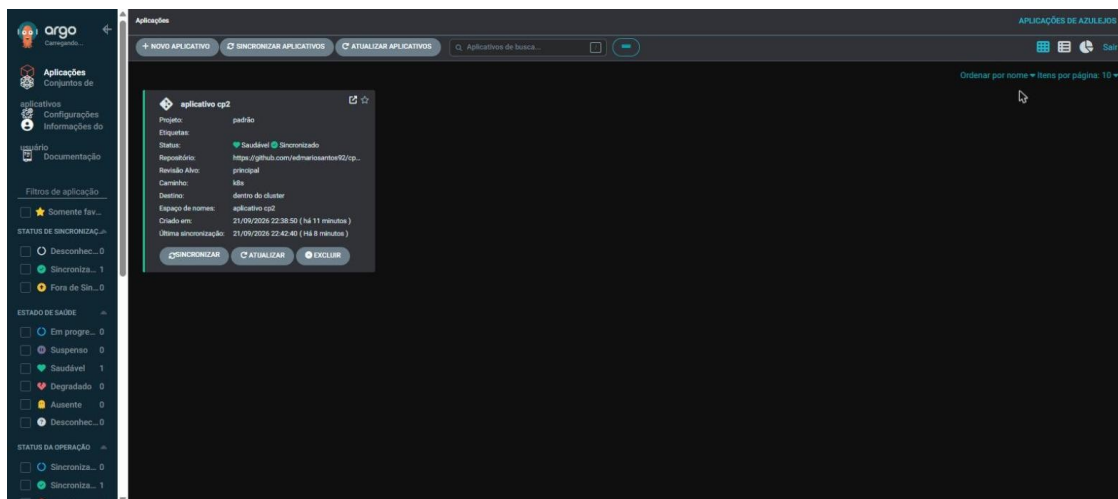
Primeira execucao do pipeline, status Success

5. Imagem publicada no Container Registry



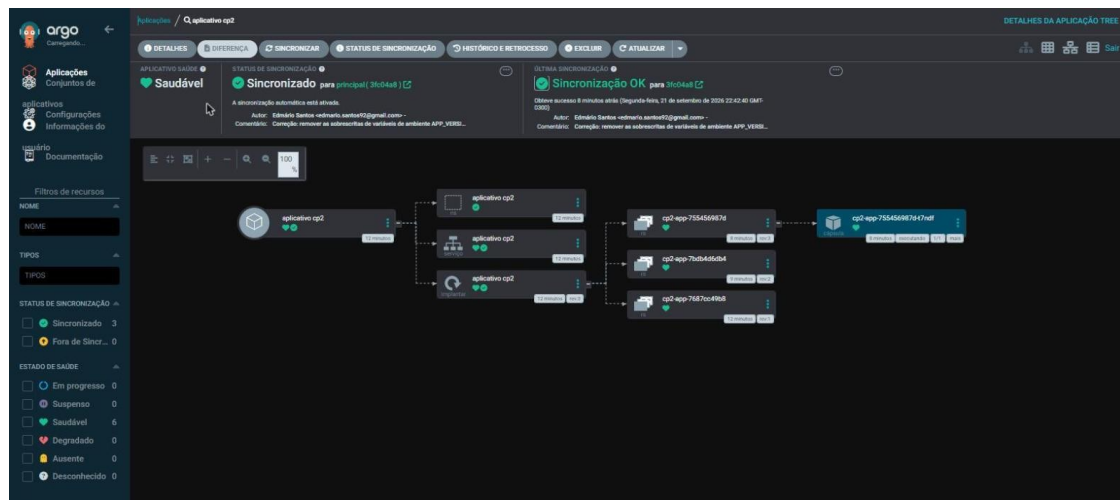
Pacote cp2-app publicado no GHCR, publico, com tags latest e por SHA

6. Argo CD instalado e funcionando



Interface do Argo CD, aplicacao cp2-app Saudavel e Sincronizada

7. Aplicacao cadastrada e sincronizada no Argo CD



Árvore de recursos: Namespace, Service e Deployment sincronizados

8. Pods e serviços da aplicacao no Kubernetes

`kubectl get pods/svc -n cp2-app - pod Running e Service NodePort`

```
$ kubectl get pods -n cp2-app -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
NOMINATED NODE   READINESS GATES
cp2-app-755456987d-t7ndf   1/1     Running   0           8m2s   10.244.0.12   minikube
<none>                 <none>

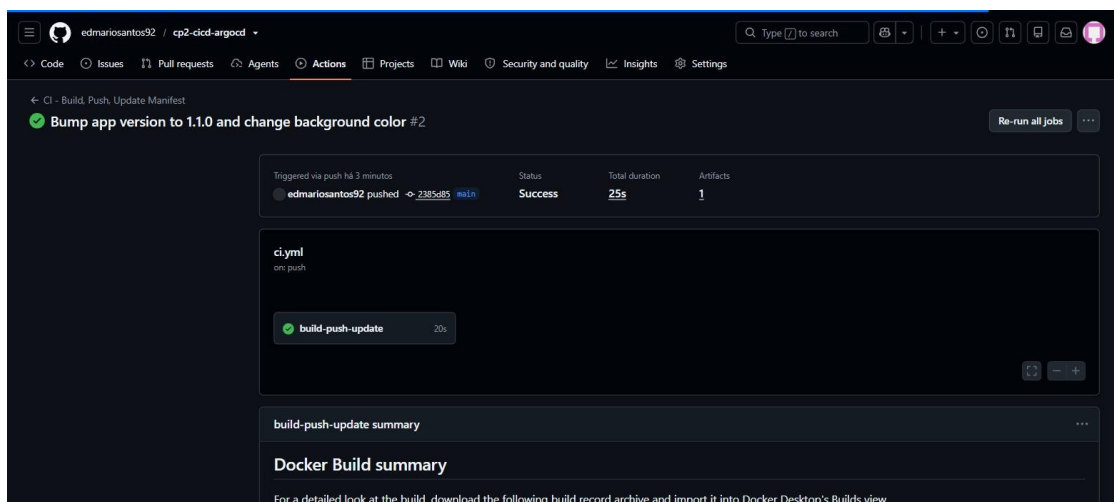
$ kubectl get svc -n cp2-app
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
cp2-app   NodePort    10.100.2.211 <none>        80:30371/TCP     11m
```

9. Aplicacao funcionando antes da alteracao



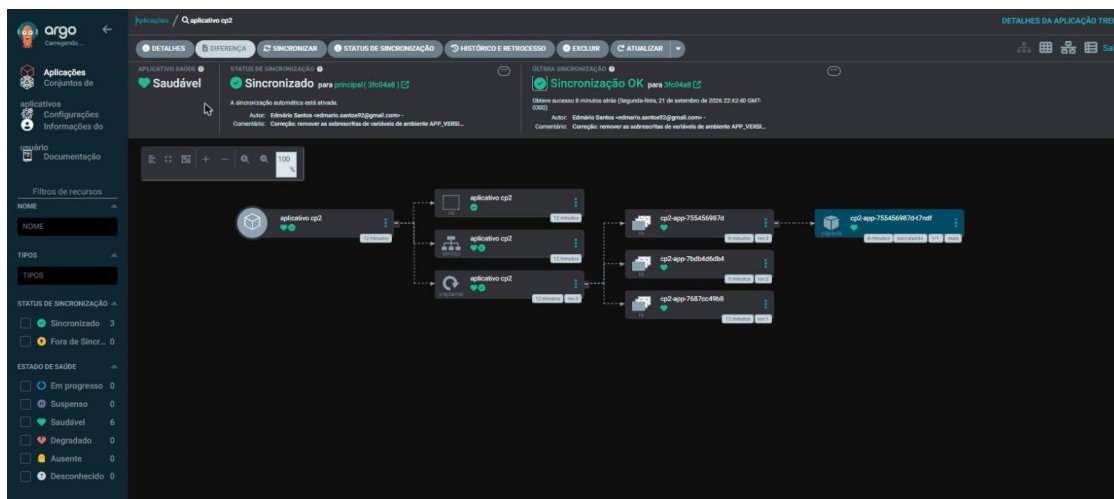
Versao 1.0.0, fundo escuro (estado inicial)

10. Nova execucao da pipeline apos a alteracao



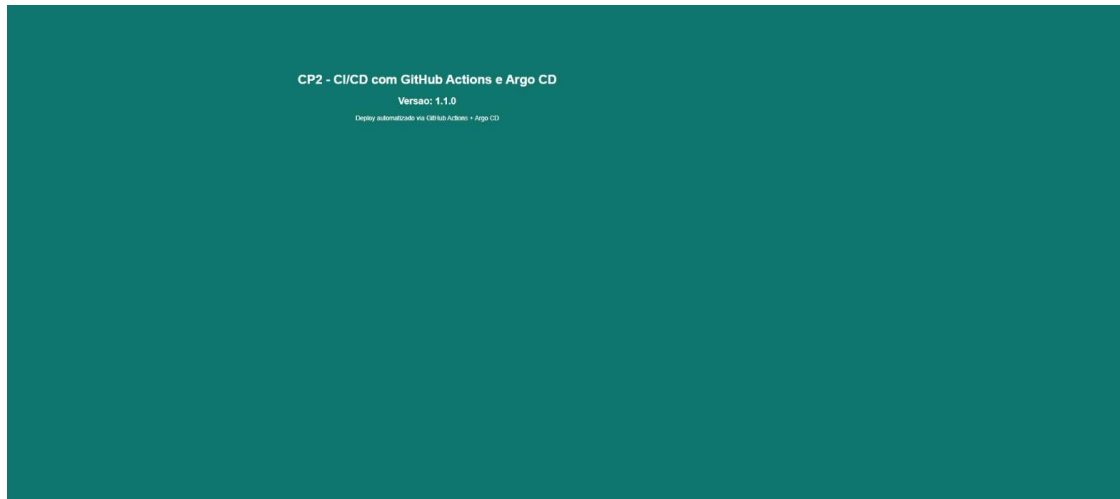
Segunda execucao do pipeline, disparada pelo bump de versao, Success

11. Atualizacao realizada pelo Argo CD



Aplicacao re-sincronizada automaticamente para a nova revisao do Git

12. Aplicacao funcionando com a nova versao



Versao 1.1.0, fundo teal (estado apos deploy automatico)

Conclusao

A atividade permitiu configurar, de ponta a ponta, um fluxo de CI/CD baseado em GitOps: o GitHub Actions cuidando da integracao continua (build e publicacao da imagem) e o Argo CD cuidando da entrega continua (sincronizacao automatica do cluster com o estado declarado no Git). O teste pratico - alterar a versao e a cor da aplicacao, dar push, e observar o pipeline, o registry e o cluster se atualizarem sozinhos - confirmou que o objetivo da atividade foi atingido: uma mudanca de codigo chega ao ambiente em producao (aqui, o cluster local) sem nenhum comando manual de deploy, apenas pela automacao configurada. As dificuldades encontradas (limite de anotacoes do kubectl, variaveis de ambiente sobrescrevendo a imagem) reforçaram na pratica conceitos importantes de Kubernetes e GitOps, como a diferenca entre configuracao da imagem e configuracao do ambiente de execucao.